Getting started with Colab and Python

Everything you need before Week 1. Fifteen minutes.

> **Read this before Week 1.** It takes fifteen minutes and it will save you several hours. Everything the course asks you to compute runs in a browser tab. Nothing needs installing on your machine.

1 What you are using, and why

Every case in this course is a **Colab notebook**: a web page that holds text and Python code in cells, running on a Google machine, not yours. You need a Google account — your `@smail.iitm.ac.in` login works — and nothing else.

The physics comes from **CoolProp**, an open-source property library carrying 136 fluids: water and steam to the IAPWS-95 reference standard, R134a, R410A, ammonia, CO_2 and the rest. When you ask for the density of saturated steam at 45 bar you get the same number a commercial code would give you.

2 The first five minutes

1. **Click the week's link.** It opens in Colab. You are signed in already if you are signed in to Gmail.
2. **Immediately do File → Save a copy in Drive.** The master notebook is read-only. Your copy appears in your own Drive, under `Colab Notebooks`, and that is the one you work in. Do this *first*, every time.
3. **Run the two setup cells.** The first installs CoolProp; the second writes the course helper module. Both take a few seconds and you only run them once per session.
4. **Then run down the notebook in order**, with Shift+Enter or the play button.

> **Cells are not independent.** A cell can only use variables that an *earlier* cell has already created. If you skip one, or run them out of order, you get `NameError`. When in doubt: Runtime → Restart and run all.

3 The runtime disconnects. This is normal.

Colab gives you a free machine and takes it back after roughly 90 minutes idle, or 12 hours of use. When it does, **every variable is gone** but your notebook text and code are safe in Drive.

Recovery is always the same: **Runtime → Restart and run all**. Because the notebooks are written to run top to bottom with no hidden state, that always works. It is worth thirty seconds; do not try to guess which cells to re-run.

4 The five errors you will actually hit

WHAT YOU SEE	WHAT IT MEANS	WHAT TO DO
`NameError: name 'x' is not defined`	You ran a cell before the one that creates `x`.	Runtime → Restart and run all.
`ModuleNotFoundError: No module named 'CoolProp'`	You skipped the install cell, or the runtime restarted.	Run the two setup cells at the top.
`ValueError: f(a) and f(b) must have different signs`	A root-finder was given a bracket with no root inside it. Usually the physics has moved outside the range you assumed.	Print the function at both ends. One of them is not what you expected, and that is the real bug.
`ValueError: Input pair variable is invalid`	A CoolProp call got two properties it cannot use together, or a state that does not exist — liquid water at 300 °C and 1 bar, for instance.	Check your units first. Pressure in Pa, temperature in K, always.
A number that is 273 out	Celsius where kelvin was wanted.	Use `am.K(30)` going in and `am.C(T)` coming out. Never write `+273.15` inside a formula.

5 How to read an error message

Python prints a **traceback**: a stack of lines ending with the actual error. Read it **from the bottom up**. The last line names the error; the line above it shows the code that raised it. Everything higher is how you got there and can usually be ignored.

```
ValueError: temperature difference must be positive at both ends
(got 50 and -3). A non-positive end means the streams cross, which
no exchanger of this configuration can do.
```

Several of the course helpers raise messages like that one. They are written to tell you *what is wrong with your engineering*, not just what is wrong with your code. Read them.

6 Units, once, properly

Everything internal is **SI**: pascals, kelvin, joules per kilogram, metres, seconds. The only exception is that arguments named `..._C` are Celsius, because that is how the briefs state them.

```
T = am.K(45)          # 45 C -> 318.15 K, at the boundary
p = 8.7e6             # pascals, not bar
print(am.C(T))        # back to Celsius, for reporting only
```

> **Bar is not an SI unit and this course does not use it internally.** Convert on the way in and on the way out. Most wrong answers in this subject are unit errors wearing a lab coat.

7 Getting your work out

Each notebook writes an Excel workbook. In Colab, click the **folder icon** on the left, find the `.xlsx`, and use its three-dot menu to download; or run the `files.download(...)` cell at the bottom.

That workbook is your deliverable. It has a Summary sheet, your results, and a Sources sheet. **Fill in the Sources sheet.** A number whose origin nobody can trace cannot be marked.

8 What good practice looks like here

- **Change one thing at a time** and re-run. Change three and you will not know which one moved the answer.

- **Check against the number in the brief** before you trust anything downstream. Every notebook tells you what it should produce.
- **Sanity-check with an energy balance.** Several notebooks print one and it should come out at machine zero. If it does not, the solver has not converged, and nothing after that point means anything.
- **State your assumptions in the report**, not in your head.

9 If you get properly stuck

Restart and run all. If it still fails, note the **last line** of the traceback and which cell raised it, and bring both. "It does not work" cannot be helped; "cell 7 raises `ValueError: f(a) and f(b) must have different signs`" can be fixed in a minute.
